Projeto Final LBD — Etapa 2: Operação, Segurança, Otimização e Auditoria

Integrantes da Equipe: Pedro Fernandes, Murilo Teles, Samuel Ribeiro, Kauã Eyke e Yan Gabriel Pereira
Domínio: Gestão Operacional de Hotelaria (`hotel_brmw`)
Data: Outubro de 2026

---

1. Revisão e Consolidação da Etapa 1

Este documento reúne a revisão da Etapa 1 (A1 — Escopo e Regras de Negócio e A5 — Verificação de Normalização) integrada às implementações físicas, transacionais e de segurança exigidas na Etapa 2.

1.1 A1 — Descrição do Domínio

O sistema atende um hotel de porte médio que precisa controlar, em uma base única, quem se hospeda, onde se hospeda, o que consome durante a estadia e em que estado se encontra cada quarto. O recorte vai do cadastro do cliente até o consumo de produtos em uma ordem de manutenção. Ficam fora do escopo faturamento fiscal, folha de pagamento, canais de venda externos e reserva de espaços de evento.

Clientes e seus vínculos

O hotel atende tanto hóspedes individuais quanto empresas que hospedam funcionários. Por isso o cliente é modelado como uma entidade genérica com dois tipos especializados: pessoa física, identificada por CPF e RG, e pessoa jurídica, identificada por CNPJ e razão social. A especialização é total e disjunta: todo cliente é de um dos dois tipos, e nunca dos dois ao mesmo tempo.

Um cliente tem um nome e, opcionalmente, um e-mail. Como é comum um hóspede deixar mais de um contato — celular pessoal, telefone do trabalho, recado —, os telefones foram extraídos para uma entidade própria, e um mesmo cliente pode ter quantos telefones quiser, inclusive nenhum.

O cliente também pode registrar dependentes, que são os acompanhantes que se hospedam com ele mas não respondem pela reserva: filhos, cônjuge, familiares. O dependente não tem existência própria no sistema — ele só faz sentido enquanto vinculado a um cliente, e é numerado dentro dele.

Quartos e reservas

Cada quarto tem um número único na numeração do prédio, um tipo (simples, duplo, suíte ou luxo) e um preço de diária. O preço é registrado no próprio quarto, não no tipo, porque o hotel pratica valores diferentes para quartos do mesmo tipo em função de andar, vista e reforma.

A reserva é o centro operacional do modelo. Toda reserva pertence a exatamente um cliente e ocupa exatamente um quarto, e registra a data prevista de entrada, a data prevista de saída e a situação em que se encontra. Uma reserva nasce pendente, é confirmada quando o pagamento ou a garantia é aceito, passa a em andamento no momento do check-in, é finalizada no check-out e pode ser cancelada antes disso. Por ser o registro que dá sentido a todo o resto, um cliente que já tenha reserva registrada não pode ser removido do cadastro.

Durante a estadia o hóspede pode solicitar serviços do catálogo do hotel — lavanderia, transfer, café no quarto, estacionamento. Cada serviço tem um nome e um preço de tabela. Um mesmo serviço é lançado uma única vez por reserva: quando o hóspede pede o mesmo item de novo, o lançamento existente tem a quantidade aumentada em vez de gerar uma segunda linha. O lançamento guarda ainda o momento em que o pedido foi feito.

Situação do quarto ao longo do tempo

Um quarto não está apenas ocupado ou livre: ele pode estar em limpeza, bloqueado por manutenção, reservado ou fora de operação. Como a administração precisa saber por quanto tempo cada quarto ficou indisponível e por qual motivo, toda alteração de situação gera um registro no histórico do quarto, com a situação anterior, a nova situação, o momento de início, o motivo e o momento de encerramento. O período corrente é justamente aquele cuja data de fim ainda está em aberto.

Manutenção, produtos e fornecedores

Cada intervenção física em um quarto é registrada como uma manutenção, sempre referente a um quarto e sempre executada por um funcionário responsável, com a data e a descrição do que foi feito.

A manutenção costuma consumir produtos do almoxarifado — lâmpadas, tintas, peças hidráulicas, itens de enxoval. Cada produto tem nome, quantidade em estoque e preço de custo, e é comprado de um único fornecedor. O consumo é registrado por manutenção, com a quantidade usada e o custo correspondente, o que permite apurar quanto cada intervenção custou em material.

Os fornecedores são pessoas jurídicas, identificadas por CNPJ, com razão social e telefone de contato.

Funcionários

Os funcionários do hotel têm nome, cargo e salário. A estrutura da equipe é hierárquica: um funcionário pode ter um supervisor, que também é funcionário, e um mesmo supervisor responde por vários subordinados. Funcionários do topo da hierarquia não têm supervisor, e ninguém supervisiona a si mesmo.

1.2 Atendimento aos Requisitos Mínimos de Complexidade

Requisito	**Mínimo exigido**	**Como o modelo atende**
Entidades	Oito, sem contar associativas	Oito principais — CLIENTE, QUARTO, RESERVA, FUNCIONARIO, SERVICO, FORNECEDOR, PRODUTO e MANUTENCAO — mais CLIENTE_TELEFONE, DEPENDENTE, HISTORICO_STATUS_QUARTO e as duas subclasses de CLIENTE.
Relacionamentos N:N com atributo próprio	Dois	`reserva_servico` (Reserva × Servico), com quantidade e data_hora_solicitacao; `item_manutencao` (Manutencao × Produto), com quantidade e custo.
Autorrelacionamento	Um	FUNCIONARIO supervisiona FUNCIONARIO, pela chave estrangeira de supervisor (RN19).
Generalização/especialização	Uma, com estratégia justificada	CLIENTE especializado em PESSOA_FISICA e PESSOA_JURIDICA. Estratégia e justificativa abaixo.
Entidade fraca	Uma, com identificação por dependência	DEPENDENTE, existencialmente dependente de CLIENTE.
Atributo temporal	Uma entidade com histórico datado	HISTORICO_STATUS_QUARTO, com situação anterior, nova

Requisito	Mínimo exigido	Como o modelo atende
		situação, início, fim e motivo (RN16).
Regras de negócio	Vinte, verificáveis e rastreáveis	RN01 a RN20.
Volume de carga	40 linhas nas principais, 100 na de maior movimento	Atendido pelo `02_carga.sql`, cuja tabela de maior movimento é `reserva_servico`.

Estratégia de mapeamento da especialização: Foi adotada a estratégia de tabela por subclasse, com junção: uma tabela base `CLIENTE` (`id_cliente`, `nome`, `email`) e duas tabelas especializadas, `PESSOA_FISICA` (`cpf`, `rg`) e `PESSOA_JURIDICA` (`cnpj`, `razao_social`), ambas tendo `id_cliente` simultaneamente como chave primária e chave estrangeira.

1.3 Tabela de Regras de Negócio (Rastreabilidade)

#	Regra	Atendida por	Mecanismo
RN01	Todo cliente é pessoa física ou pessoa jurídica, nunca ambos.	Banco (parcial) + Consulta	PK/FK em PESSOA_FISICA e PESSOA_JURIDICA impede duplicação na mesma subclasse.
RN02	CPF e CNPJ são únicos no cadastro.	Banco	`uq_pessoa_fisica_cpf` e `uq_pessoa_juridica_cnpj`.
RN03	O e-mail do cliente, quando informado, não pode se repetir.	Banco	`uq_cliente_email`.
RN04	O telefone do cliente é opcional.	Banco	Entidade CLIENTE_TELEFONE com cardinalidade mínima zero.
RN05	O dependente só existe vinculado a um cliente.	Banco (parcial) + Aplicação	`fk_dependente_cliente` com ON DELETE CASCADE.
RN06	O número do quarto é único no hotel.	Banco	`uq_quarto_numero`.
RN07	O tipo do quarto é SIMPLES, DUPLO, SUÍTE ou LUXO.	Banco	`ck_quarto_tipo`.

#	Regra	Atendida por	Mecanismo
RN08	O preço da diária é sempre maior que zero.	Banco	`ck_quarto_preco_diaria` (`preco_diaria > 0`).
RN09	Toda reserva pertence a exatamente um cliente e um quarto.	Banco	`fk_reserva_cliente` e `fk_reserva_quarto` (NOT NULL).
RN10	A data de checkout é posterior à data de checkin.	Banco	`ck_reserva_datas` (`data_checkout > data_checkin`).
RN11	Status da reserva: PENDENTE, CONFIRMADA, EM_ANDAMENTO, FINALIZADA ou CANCELADA.	Banco	`ck_reserva_status`.
RN12	O valor total da reserva nunca é negativo.	Consulta	Calculado dinamicamente das diárias somadas aos serviços.
RN13	Cliente com reserva registrada não pode ser excluído.	Banco	`fk_reserva_cliente` (ON DELETE RESTRICT).
RN14	Um mesmo serviço é lançado uma única vez por reserva.	Banco	Chave primária composta `pk_reserva_servico` (`id_reserva`, `id_servico`).
RN15	A quantidade de serviço solicitada é de no mínimo 1.	Banco	`ck_reserva_servico_quantidade` (`quantidade >= 1`).
RN16	Alteração de situação de quarto gera registro histórico datado.	Banco (parcial) + Aplicação	Mantido via `HISTORICO_STATUS_QUARTO` e triggers de auditoria.
RN17	Data de fim do histórico, quando informada, é posterior ao início.	Banco	`ck_historico_datas`.
RN18	Toda manutenção é referente a um quarto e executada por funcionário.	Banco	`fk_manutencao_quarto` e `fk_manutencao_funcionario`.
RN19	Funcionário tem no máximo um supervisor e não supervisiona a si mesmo.	Banco	`fk_funcionario_supervisor` e

#	Regra	Atendida por	Mecanismo
			`ck_funcionario_supervisor`.
RN20	Todo produto pertence a um fornecedor; estoque e custo não são negativos.	Banco	`fk_produto_fornecedor`, `ck_produto_estoque`, `ck_produto_custo`.

1.4 A5 — Verificação de Normalização

O esquema relacional encontra-se na **Terceira Forma Normal (3FN)** e atinge a **Forma Normal de Boyce-Codd (BCNF)** em todas as 15 relações.

- **1FN:** Atributos multivalorados (como telefones e dependentes) foram extraídos para tabelas próprias. Todos os atributos são atômicos.
- **2FN:** Não existem dependências parciais. Em `RESERVA_SERVICO` e `ITEM_MANUTENCAO` (únicas de chave composta), todos os atributos não-chave dependem da totalidade da chave primária.
- **3FN / BCNF:** Não existem dependências transitivas. Todos os determinantes são chaves candidatas.

---

2. Artefatos de Infraestrutura e Autenticação da API (`04_api.sql`)

O script `04_api.sql` estende o modelo para suporte à autenticação segura (com hashing de senhas), estrutura de auditoria centralizada e o usuário de banco de dados com privilégios estritos para a aplicação.

```sql
-- ============================================================================
-- 04_api.sql  -- Preparação do banco para a API (rodar DEPOIS de 01, 02 e 03)
--   * login do funcionário (senha só como hash bcrypt, nunca em texto)
--   * tabela de auditoria
--   * usuário do MySQL usado pela API, com privilégios mínimos
-- ============================================================================
USE hotel_brmw;
```

```
ALTER TABLE funcionario
    ADD COLUMN login      VARCHAR(40)  NULL,
    ADD COLUMN senha_hash VARCHAR(100) NULL,
    ADD COLUMN papel      VARCHAR(12)  NULL,
    ADD COLUMN ativo      TINYINT(1)   NOT NULL DEFAULT 1,
    ADD CONSTRAINT uq_funcionario_login UNIQUE (login),
    ADD CONSTRAINT ck_funcionario_papel CHECK (papel IS NULL OR papel IN
('GERENTE','RECEPCAO'));

CREATE TABLE auditoria (
    id_auditoria BIGINT       NOT NULL AUTO_INCREMENT,
    data_hora    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    usuario      VARCHAR(60)  NOT NULL,
    papel        VARCHAR(20)  NOT NULL,
    acao         VARCHAR(30)  NOT NULL,
    tabela       VARCHAR(40)  NOT NULL,
    registro     VARCHAR(40)  NOT NULL,
    detalhe      VARCHAR(500)     NULL,
    CONSTRAINT pk_auditoria PRIMARY KEY (id_auditoria)
) ENGINE = InnoDB;
CREATE INDEX idx_auditoria_data ON auditoria (data_hora);

-- Usuário da API (troque o placeholder e use a mesma senha em backend/.env)
CREATE USER IF NOT EXISTS 'hotel_app'@'localhost' IDENTIFIED BY
'TROQUE_ANTES_DE_EXECUTAR';

-- Permissões de Leitura (Princípio do menor privilégio)
GRANT SELECT ON hotel_brmw.cliente TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.pessoa_fisica TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.pessoa_juridica TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.dependente TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.funcionario TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.quarto TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.historico_status_quarto TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.reserva TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.servico TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.reserva_servico TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.fornecedor TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.produto TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.manutencao TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.item_manutencao TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.auditoria TO 'hotel_app'@'localhost';

-- Permissões de Escrita Operacional
GRANT INSERT ON hotel_brmw.cliente                TO 'hotel_app'@'localhost';
GRANT INSERT ON hotel_brmw.pessoa_fisica          TO 'hotel_app'@'localhost';
GRANT INSERT ON hotel_brmw.pessoa_juridica        TO 'hotel_app'@'localhost';
GRANT INSERT, UPDATE ON hotel_brmw.reserva        TO 'hotel_app'@'localhost';
```

```
GRANT UPDATE ON hotel_brmw.quarto                  TO 'hotel_app'@'localhost';
GRANT INSERT, UPDATE ON hotel_brmw.historico_status_quarto TO
'hotel_app'@'localhost';
GRANT INSERT ON hotel_brmw.auditoria               TO 'hotel_app'@'localhost';

-- Permissões para CRUDs administrativos (restritos à GERENTE na aplicação)
GRANT INSERT, UPDATE, DELETE ON hotel_brmw.quarto      TO 'hotel_app'@'localhost';
GRANT INSERT, UPDATE, DELETE ON hotel_brmw.servico     TO 'hotel_app'@'localhost';
GRANT INSERT, UPDATE, DELETE ON hotel_brmw.produto     TO 'hotel_app'@'localhost';
GRANT INSERT, UPDATE, DELETE ON hotel_brmw.fornecedor  TO 'hotel_app'@'localhost';
GRANT SELECT ON hotel_brmw.vw_relatorio_hotel          TO 'hotel_app'@'localhost';
FLUSH PRIVILEGES;
```

---

3. Artefatos de Visões, Triggers, Perfis e Otimização (`05_etapa2.sql`)

3.1 B1 — Visão do Relatório Consolidado

A visão `vw_relatorio_hotel` consolida os indicadores operacionais e financeiros por categoria de quarto.

```
SQL
USE hotel_brmw;

-- B1: visão que alimenta a tela de relatório consolidado.
CREATE OR REPLACE VIEW vw_relatorio_hotel AS
SELECT q.tipo AS tipo_quarto,
       COUNT(DISTINCT q.id_quarto) AS total_quartos,
       COUNT(DISTINCT CASE WHEN q.situacao_atual = 'DISPONIVEL' THEN q.id_quarto
END) AS quartos_disponiveis,
       COUNT(DISTINCT CASE WHEN q.situacao_atual = 'OCUPADO' THEN q.id_quarto END)
AS quartos_ocupados,
       COUNT(DISTINCT CASE WHEN r.status <> 'CANCELADA' THEN r.id_reserva END) AS
reservas_nao_canceladas,
       COALESCE(SUM(CASE WHEN r.status NOT IN ('CANCELADA') THEN r.valor_total
ELSE 0 END), 0) AS valor_reservas,
       ROUND(AVG(CASE WHEN r.status NOT IN ('CANCELADA') THEN r.valor_total END),
2) AS ticket_medio
  FROM quarto q
  LEFT JOIN reserva r ON r.id_quarto = q.id_quarto
 GROUP BY q.tipo;
```

3.2 B6 — Gatilhos de Auditoria Automatizada

Gatilhos para auditoria de modificações diretas nas tabelas `reserva` e `quarto`, capturando o estado do registro pré e pós-alteração via JSON.

```SQL
-- B6: triggers para auditar alterações de reservas e quartos no próprio banco.
DELIMITER $$
CREATE TRIGGER trg_aud_reserva_update
AFTER UPDATE ON reserva
FOR EACH ROW
BEGIN
  INSERT INTO auditoria (data_hora, usuario, papel, acao, tabela, registro,
detalhe)
  VALUES (NOW(), CURRENT_USER(), 'BANCO', 'UPDATE', 'reserva', CAST(NEW.id_reserva
AS CHAR),
    CONCAT('ANTES=', JSON_OBJECT('status', OLD.status, 'valor_total',
OLD.valor_total,
      'data_checkin', OLD.data_checkin, 'data_checkout', OLD.data_checkout),
      '; DEPOIS=', JSON_OBJECT('status', NEW.status, 'valor_total',
NEW.valor_total,
      'data_checkin', NEW.data_checkin, 'data_checkout', NEW.data_checkout)));
END$$

CREATE TRIGGER trg_aud_quarto_update
AFTER UPDATE ON quarto
FOR EACH ROW
BEGIN
  IF OLD.situacao_atual <> NEW.situacao_atual OR OLD.preco_diaria <>
NEW.preco_diaria THEN
    INSERT INTO auditoria (data_hora, usuario, papel, acao, tabela, registro,
detalhe)
    VALUES (NOW(), CURRENT_USER(), 'BANCO', 'UPDATE', 'quarto', CAST(NEW.id_quarto
AS CHAR),
      CONCAT('ANTES=', JSON_OBJECT('situacao_atual', OLD.situacao_atual,
'preco_diaria', OLD.preco_diaria),
        '; DEPOIS=', JSON_OBJECT('situacao_atual', NEW.situacao_atual,
'preco_diaria', NEW.preco_diaria)));
  END IF;
END$$
DELIMITER ;
```

3.3 B5 — Segurança, Roles e Princípio do Menor Privilégio

Configuração de três perfis no MySQL 8.0 e demonstração explícita do comando `REVOKE`.

```sql
-- B5: três perfis do banco usando roles MySQL 8.0.
CREATE ROLE IF NOT EXISTS 'hotel_leitura', 'hotel_recepcao', 'hotel_gerencia';

-- Perfil de Leitura: Acesso exclusivo à visão agregada
GRANT SELECT ON hotel_brmw.vw_relatorio_hotel TO 'hotel_leitura';

-- Perfil de Recepção: Acesso a dados operacionais
GRANT SELECT ON hotel_brmw.reserva TO 'hotel_recepcao';
GRANT SELECT ON hotel_brmw.cliente TO 'hotel_recepcao';
GRANT SELECT ON hotel_brmw.pessoa_fisica TO 'hotel_recepcao';
GRANT SELECT ON hotel_brmw.pessoa_juridica TO 'hotel_recepcao';
GRANT SELECT ON hotel_brmw.quarto TO 'hotel_recepcao';
GRANT INSERT, UPDATE ON hotel_brmw.reserva TO 'hotel_recepcao';
GRANT UPDATE ON hotel_brmw.quarto TO 'hotel_recepcao';

-- Demonstração explícita de REVOKE
GRANT UPDATE ON hotel_brmw.produto TO 'hotel_recepcao';
REVOKE UPDATE ON hotel_brmw.produto FROM 'hotel_recepcao';

-- Perfil de Gerência: Acesso total ao schema e relatórios de auditoria
GRANT SELECT, INSERT, UPDATE, DELETE ON hotel_brmw.* TO 'hotel_gerencia';
GRANT SELECT ON hotel_brmw.auditoria TO 'hotel_gerencia';
```

3.4 B4 — Otimização de Consultas e Análise de Desempenho

```sql
-- B4: Índices criados para otimização de consultas frequentes
CREATE INDEX idx_reserva_quarto_status_periodo
    ON reserva (id_quarto, status, data_checkin, data_checkout);

CREATE INDEX idx_reserva_status_checkin
    ON reserva (status, data_checkin);

CREATE INDEX idx_reserva_servico_reserva
    ON reserva_servico (id_reserva, id_servico);
```

Roteiro de Medição de Desempenho

Para avaliar o ganho computacional gerado pelos índices criados, o plano de execução (`EXPLAIN ANALYZE`) deve ser obtido para cada uma das consultas a seguir:

1. **Consulta de Ocupação e Disponibilidade por Quarto/Período:**

```sql
EXPLAIN ANALYZE
SELECT id_reserva
  FROM reserva
 WHERE id_quarto = 1
   AND status = 'CONFIRMADA'
   AND data_checkin < '2026-12-01'
   AND '2026-11-01' < data_checkout;
```

2. **Agrupamento de Reservas por Intervalo de Check-in:**

```sql
EXPLAIN ANALYZE
SELECT status, COUNT(*)
  FROM reserva
 WHERE data_checkin BETWEEN '2026-01-01' AND '2026-12-31'
 GROUP BY status
HAVING COUNT(*) > 1;
```

3. **Volumetria de Serviços Solicitados por Reserva:**

```sql
EXPLAIN ANALYZE
SELECT id_reserva, COUNT(*)
  FROM reserva_servico
 GROUP BY id_reserva;
```